FS Exam Prep — Phase 1 Build & Smoke Test: Agent Prompt

Version · v1.0.1 — 2026-10-24 (v1.0.0 same day; revised: applicationId decided, Step 3 now applies it)

For · The coding agent working on Charlie's Windows PC (Android Studio project open, terminal access, his Android phone connected). **Not** a Google AI Studio prompt — this phase is local builds and a physical device.

Use · Attach the two files listed below, then paste the prompt verbatim.

Base · Repo @ `51c7823`; spec = Phase 1 Spec v1.0.1 (attached).

Attach these files with the prompt (PDFs):

1. `2026-10-24_fs-phase1-build-spec_v1.0.1.pdf` — the phase spec, including the §Smoke script (authority for every step).
2. `2026-10-22_fs-app-launch-roadmap_v1.0.0.pdf` — Phase 1 context and what comes after.

TASK: Phase 1 for the FS Exam Prep Android app — first verified release build, signing setup, and an 11-step device smoke test. The attached Phase 1 Spec is the authority; this prompt is the run order. Work on Charlie's PC, in the Android Studio project (FS-Exam-Prep-Mobile-App, base commit 51c7823).

RULES THAT OVERRIDE EVERYTHING ELSE: - STOP means stop and ask Charlie. Do not guess past a STOP. - Never ask Charlie to tell you a password in chat, never type, store, log, or echo keystore passwords. Keystore password entry is Charlie's hands only. - Do not fix app bugs in this phase. If the smoke test exposes one, record it (step, what happened, stack trace) and continue with the next independent step. - No feature changes, no dependency changes, no theme changes.

STEP 1 — Repo safety (before anything else). The repo .gitignore does NOT cover keystore files, and the Gradle release config defaults to ${rootDir}/my-upload-key.jks. A key created at the project root

could be committed by accident. - Append to .gitignore: *.jks *.keystore keystore.properties my-upload-key.jks - Verify: `git check-ignore my-upload-key.jks` must print the filename, and `git status` must stay clean of it. - Commit ONLY this change: "chore: ignore keystore files".

STEP 2 — Version. In app/build.gradle.kts set versionName "0.9.0" and versionCode 2. Change nothing else in that file — especially not the signingConfigs blocks (their "do not modify" comments stand; signing is wired in Step 4, not by editing them). Commit: "chore: version 0.9.0 (code 2)".

STEP 3 — applicationId (decided — apply it). Charlie has decided the final applicationId: com.crieck.prep.fs (family pattern com.crieck.prep.*). In app/build.gradle.kts, defaultConfig, change ONLY the applicationId line, from com.aistudio.fsexamprep.kqzrxp to com.crieck.prep.fs. - Do NOT change the namespace line (it stays com.example — the namespace is the code package, not the store identity, and changing it breaks the build). - Commit separately: "chore: applicationId com.crieck.prep.fs".

STEP 4 — Upload key (Charlie's hands). - Charlie creates the key via Android Studio: Build > Generate Signed App Bundle/APK > Create new... — alias "upload", validity 25+ years. Passwords are entered by Charlie alone. - Preferred location: a secure folder OUTSIDE the repo. Only if it must live at the project root as my-upload-key.jks, Step 1 already makes that safe. - Confirm Charlie has backed up the .jks and passwords to his password manager before continuing. If he hasn't, pause here.

STEP 5 — Build. Wire the release signing via the environment route the Gradle config already reads (KEYSTORE_PATH, STORE_PASSWORD, KEY_PASSWORD) or the Android Studio wizard. Then run: gradlew.bat bundleRelease gradlew.bat assembleRelease Report success/failure and any warnings. A failed build ends the phase — report the errors verbatim, do not start patching.

STEP 6 — Smoke test on Charlie's phone. Install the release APK fresh (uninstall any existing copy first). Walk S1–S11 from the attached spec's Smoke section, in order. Capture a screenshot at every step into a folder named smoke-test-0.9.0 OUTSIDE the repo, and keep a logcat recording of the whole session. Charlie performs the in-app taps; you drive adb, screenshots, and the checklist. For

S10, force-stop and relaunch via adb. For S11, scan the logcat for "FATAL EXCEPTION" on the app package.

STEP 7 — Report. Deliver: (1) per-step PASS/FAIL for S1–S11 with the screenshot filenames; (2) any failures with exact behavior and stack traces; (3) build artifact paths for the AAB and APK; (4) the versionName/versionCode/applicationId actually built — the applicationId must read com.crieck.prep.fs; (5) git log showing the Step 1–3 commits. Push nothing except the Step 1–3 commits, and only if Charlie confirms.

After the run — Charlie's checks

1. The applicationId is decided (com.crieck.prep.fs), so the Step 5 AAB — once built with it — is the real upload candidate for the Phase 3 internal track.
2. Any smoke failures come back to Muse for a Studio fix prompt; don't let the PC agent patch app code ad hoc.
3. Muse verifies the two commits (gitignore, version) on GitHub on the word.